

Bases de Dados Relacionais e SQL II

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercícios

Exercício 1: Agregações

Use a base de dados `university.db` criada na aula anterior (ou recree-a usando a solução fornecida).

1. Conte quantos estudantes existem na base de dados.
 2. Conte quantos estudantes existem no departamento de `Computer Science`.
 3. Calcule a idade média de todos os estudantes.
 4. Encontre a idade mínima e máxima na base de dados.
 5. Mostre o número de estudantes por departamento usando `GROUP BY`.
-

Exercício 2: Subqueries e Joins Avançados

1. Encontre os nomes dos estudantes que estão num departamento que tem mais de 2 estudantes.
 2. Selecione os departamentos que não têm nenhum estudante (use uma subquery com `NOT IN`).
 3. Crie uma vista (VIEW) chamada `student_details` que junte as tabelas de estudantes e departamentos.
-

Exercício 3: Desempenho e Índices

1. Explique a diferença entre um `Sequential Scan` e um `Index Scan`.
2. Crie um índice na coluna `name` da tabela `students`.
3. No SQLite, use `EXPLAIN QUERY PLAN` seguido de uma consulta `SELECT` para ver se o índice está a ser utilizado.

```
EXPLAIN QUERY PLAN SELECT * FROM students WHERE name = 'Alice';
```

Exercício 4: Trabalhar com Conjuntos de Dados Grandes (Titanic)

1. Importe o ficheiro `dataset/titanic.csv` para uma nova base de dados SQLite chamada `titanic.db`.
 - Dica: Use `.mode csv` e `.import dataset/titanic.csv passengers` dentro do `sqlite3`.
 2. Escreva consultas para responder:
 - Qual foi a taxa de sobrevivência dos passageiros?
 - Qual foi a idade média dos sobreviventes vs. não sobreviventes?
 - Qual a classe de passageiro (`Pclass`) que teve o maior número de sobreviventes?
 - Será que as mulheres e crianças tiveram realmente uma maior probabilidade de sobrevivência? (Agrupe por Sexo e uma categoria de Idade calculada).
-

Exercício 5: Desafio de Segurança (SQL Injection)

1. Observe este código Python:

```
user_id = input("Introduza o ID do utilizador: ")
cursor.execute(f"SELECT * FROM users WHERE id = {user_id}")
```

2. O que poderia um utilizador mal-intencionado escrever para apagar a tabela users inteira?
3. Reescreva o código para ser seguro usando um prepared statement.